

MEMORIA DE ANÁLISIS

Base de Datos para NexShop Group S.A.

Alumno: Daniel Zanfaño García

1. Descripción de la empresa

NexShop Group S.A. es una empresa de distribución y venta al por menor fundada en 2015 con sede en Valencia. Opera bajo dos líneas de negocio diferenciadas: una tienda online (nexshop.es) y una red de tres tiendas físicas situadas en Valencia, Madrid y Barcelona.

Aunque ambas líneas comparten el mismo catálogo de productos, funcionan de forma casi independiente y tienen equipos distintos. La empresa cuenta con 47 empleados repartidos entre las distintas sedes.

La sede central de Valencia coordina los pedidos a proveedores y la reposición de stock en todas las ubicaciones, a través del equipo de logística y el departamento de compras.

2. Identificación de entidades

A partir del análisis exhaustivo de las tres fuentes proporcionadas (descripción general, acta de reunión y correos internos), se han identificado las siguientes entidades. Para cada una se indica el motivo de su inclusión.

2.1 Entidades principales y justificación

SEDE

Motivo: Lo pide el cliente

NexShop opera en múltiples ubicaciones físicas (Valencia, Madrid, Barcelona) y un almacén central. Es necesario modelarlas para gestionar stock, empleados y transferencias por ubicación.

EMPLEADO

Motivo: Lo pide el cliente

El cliente requiere saber qué empleado realizó cada venta presencial, qué agente gestiona cada ticket y quién autorizó cada transferencia. Atributos solicitados explícitamente: nombre, DNI, email corporativo, fecha de incorporación y sede asignada.

CATEGORIA

Motivo: Lo pide el cliente

El catálogo está organizado en categorías y subcategorías jerárquicas (ej. Informática → Portátiles → Gaming). Se modela con autoreferencia para soportar la jerarquía.

PRODUCTO

Motivo: Lo pide el cliente

Entidad central del negocio. Más de 2.000 referencias con PVP que puede variar con el tiempo. Un producto pertenece a una única subcategoría.

HISTORIAL_PRECIO

Motivo: Lo pide el cliente

El PVP de un producto puede variar con el tiempo y el sistema debe mostrar el historial completo. Se separa del producto para no perder datos históricos.

PROMOCION

Motivo: Lo pide el cliente

El departamento de marketing lanza promociones con descuento porcentual sobre determinados productos durante un rango de fechas. Un producto puede tener varias promociones a lo largo del año.

PROVEEDOR

Motivo: Lo pide el cliente

Los proveedores suministran productos al almacén central. Cada proveedor tiene un representante comercial de NexShop asignado.

CONDICION_PROVEEDOR

Motivo: Lo pide el cliente

Para cada combinación producto-proveedor se negocia un precio de coste y un plazo de entrega. Estos datos cambian periódicamente y es importante guardar el histórico.

CLIENTE

Motivo: Lo pide el cliente

Los clientes se registran en la plataforma online. También existen clientes anónimos (compras presenciales sin registro). Se modela de forma que permita la vinculación posterior.

DIRECCION

Motivo: Lo pide el cliente

Un cliente puede tener múltiples direcciones guardadas (domicilio, trabajo, otras), con calle, número, piso, CP, ciudad y país.

PEDIDO_ONLINE

Motivo: Lo pide el cliente

Proceso de compra online vinculado a un cliente registrado y a una dirección de entrega. Puede generar varios envíos parciales.

LINEA_PEDIDO

Motivo: Lo propongo yo

Tabla intermedia necesaria para resolver la relación N:M entre PEDIDO_ONLINE y PRODUCTO, almacenando cantidad y precio unitario en el momento de la compra.

ENVIO

Motivo: Lo pide el cliente

Un pedido puede generar varios envíos parciales desde distintos almacenes. Cada envío tiene número de seguimiento, transportista y fecha estimada de entrega.

LINEA_ENVIO

Motivo: Lo propongo yo

Tabla intermedia que indica qué productos y qué cantidades contiene cada envío, permitiendo los envíos parciales descritos por David Cano.

VENTA_PRESENCIAL

Motivo: Lo pide el cliente

Proceso de venta en tienda física, distinto al pedido online. Se genera un ticket de venta vinculado al empleado que realizó la venta. El cliente puede ser anónimo.

LINEA_VENTA

Motivo: Lo propongo yo

Tabla intermedia N:M entre VENTA_PRESENCIAL y PRODUCTO. Almacena los productos y cantidades de cada venta presencial.

DEVOLUCION_PRESENCIAL

Motivo: Lo pide el cliente

Cuando un cliente devuelve algo comprado en tienda, se genera un documento de devolución vinculado al ticket original.

STOCK_UBICACION

Motivo: Lo pide el cliente

El stock de cada producto se controla por ubicación. Cada tienda y el almacén central tienen su propio nivel de stock para cada referencia.

TRANSFERENCIA_STOCK

Motivo: Lo pide el cliente

Registra las transferencias internas de stock entre sedes con fecha, origen, destino, producto, cantidad y el empleado que la autorizó.

TICKET_INCIDENCIA

Motivo: Lo pide el cliente

Cuando un cliente contacta con atención al cliente se abre un ticket con asunto, descripción, fecha de apertura, estado, agente asignado y opcionalmente vinculado a un pedido.

VALORACION

Motivo: Lo pide el cliente

Los clientes registrados pueden puntuar (1-5) y comentar productos que hayan comprado. Solo una valoración por cliente y producto. Se distingue si está verificada (compra confirmada) o no.

MOVIMIENTO_PUNTOS

Motivo: Lo pide el cliente

Registro de cada movimiento de puntos del programa de fidelización: cuándo, en qué pedido, cuántos puntos y si fueron ganados o canjeados. El saldo se calcula siempre desde este histórico.

3. Atributos de cada entidad

A continuación se detallan los atributos de cada entidad, indicando la clave primaria (PK), claves foráneas (FK) y restricciones relevantes.

SEDE

Campo	Tipo	Restricciones
id_sede	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(100)	NOT NULL
tipo	ENUM('tienda','almacen')	NOT NULL
ciudad	VARCHAR(100)	NOT NULL
direccion	VARCHAR(255)	

EMPLEADO

Campo	Tipo	Restricciones
id_empleado	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(100)	NOT NULL
apellidos	VARCHAR(150)	NOT NULL
dni	VARCHAR(20)	UNIQUE NOT NULL
email_corporativo	VARCHAR(150)	UNIQUE NOT NULL
fecha_incorporacion	DATE	NOT NULL
id_sede	INT	FK → SEDE
rol	VARCHAR(50)	NOT NULL

CATEGORIA

Campo	Tipo	Restricciones
id_categoria	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(100)	NOT NULL
id_categoria_padre	INT	FK → CATEGORIA (NULL si es raíz)

PRODUCTO

Campo	Tipo	Restricciones
id_producto	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(200)	NOT NULL
descripcion	TEXT	
pvp_actual	DECIMAL(10,2)	NOT NULL
id_subcategoria	INT	FK → CATEGORIA, NOT NULL
activo	TINYINT(1)	DEFAULT 1

HISTORIAL_PRECIO

Campo	Tipo	Restricciones
id_historial	INT	PK, AUTO_INCREMENT
id_producto	INT	FK → PRODUCTO
pvp	DECIMAL(10,2)	NOT NULL
fecha_inicio	DATE	NOT NULL
fecha_fin	DATE	NULL si es el precio actual

PROMOCION

Campo	Tipo	Restricciones
id_promocion	INT	PK, AUTO_INCREMENT
id_producto	INT	FK → PRODUCTO
descuento_porcentual	DECIMAL(5,2)	NOT NULL, CHECK > 0 AND <= 100
fecha_inicio	DATE	NOT NULL
fecha_fin	DATE	NOT NULL
descripcion	VARCHAR(255)	

PROVEEDOR

Campo	Tipo	Restricciones
id_proveedor	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(150)	NOT NULL
email	VARCHAR(150)	
telefono	VARCHAR(30)	
id_representante	INT	FK → EMPLEADO

CONDICION_PROVEEDOR

Campo	Tipo	Restricciones
id_condicion	INT	PK, AUTO_INCREMENT
id_proveedor	INT	FK → PROVEEDOR
id_producto	INT	FK → PRODUCTO
precio_coste	DECIMAL(10,2)	NOT NULL
plazo_entrega_dias	INT	NOT NULL
fecha_inicio	DATE	NOT NULL
fecha_fin	DATE	NULL si es la condición actual

CLIENTE

Campo	Tipo	Restricciones
id_cliente	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(100)	NULL (puede ser anónimo)
apellidos	VARCHAR(150)	NULL
email	VARCHAR(150)	UNIQUE, NULL si anónimo
password_hash	VARCHAR(255)	NULL si anónimo
fecha_nacimiento	DATE	NULL
registrado	TINYINT(1)	DEFAULT 0
fecha_registro	DATETIME	NULL

DIRECCION

Campo	Tipo	Restricciones
id_direccion	INT	PK, AUTO_INCREMENT
id_cliente	INT	FK → CLIENTE
tipo	ENUM('domicilio', 'trabajo', 'otra')	NOT NULL
calle	VARCHAR(200)	NOT NULL
numero	VARCHAR(10)	
piso	VARCHAR(20)	
codigo_postal	VARCHAR(10)	NOT NULL
ciudad	VARCHAR(100)	NOT NULL
pais	VARCHAR(100)	NOT NULL

PEDIDO_ONLINE

Campo	Tipo	Restricciones
id_pedido	INT	PK, AUTO_INCREMENT
id_cliente	INT	FK → CLIENTE
id_direccion_entrega	INT	FK → DIRECCION
fecha_pedido	DATETIME	NOT NULL
estado	ENUM('pendiente', 'en_proceso', 'enviado', 'entregado', 'cancelado')	NOT NULL
total	DECIMAL(10,2)	NOT NULL
puntos_descuento	INT	DEFAULT 0

LINEA_PEDIDO

Campo	Tipo	Restricciones
id_linea	INT	PK, AUTO_INCREMENT
id_pedido	INT	FK → PEDIDO_ONLINE
id_producto	INT	FK → PRODUCTO
cantidad	INT	NOT NULL, CHECK > 0
precio_unitario	DECIMAL(10,2)	NOT NULL
descuento_aplicado	DECIMAL(5,2)	DEFAULT 0

ENVIO

Campo	Tipo	Restricciones
id_envio	INT	PK, AUTO_INCREMENT
id_pedido	INT	FK → PEDIDO_ONLINE
id_sede_origen	INT	FK → SEDE

numero_seguimiento	VARCHAR(100)	UNIQUE
transportista	VARCHAR(100)	
fecha_estimada	DATE	
fecha_entrega_real	DATE	NULL
estado	ENUM('preparando','en_camino','en_tregado')	NOT NULL

LINEA_ENVIO

Campo	Tipo	Restricciones
id_linea_envio	INT	PK, AUTO INCREMENT
id_envio	INT	FK → ENVIO
id_producto	INT	FK → PRODUCTO
cantidad	INT	NOT NULL, CHECK > 0

VENTA_PRESENCIAL

Campo	Tipo	Restricciones
id_venta	INT	PK, AUTO INCREMENT
id_sede	INT	FK → SEDE
id_empleado	INT	FK → EMPLEADO
id_cliente	INT	FK → CLIENTE, NULL si anónimo
fecha_venta	DATETIME	NOT NULL
total	DECIMAL(10,2)	NOT NULL

LINEA_VENTA

Campo	Tipo	Restricciones
id_linea_venta	INT	PK, AUTO INCREMENT
id_venta	INT	FK → VENTA_PRESENCIAL
id_producto	INT	FK → PRODUCTO
cantidad	INT	NOT NULL, CHECK > 0
precio_unitario	DECIMAL(10,2)	NOT NULL

DEVOLUCION_PRESENCIAL

Campo	Tipo	Restricciones
id_devolucion	INT	PK, AUTO INCREMENT
id_venta	INT	FK → VENTA_PRESENCIAL
id_empleado	INT	FK → EMPLEADO
fecha_devolucion	DATETIME	NOT NULL
motivo	TEXT	
total devuelto	DECIMAL(10,2)	NOT NULL

STOCK_UBICACION

Campo	Tipo	Restricciones
id_stock	INT	PK, AUTO INCREMENT
id_producto	INT	FK → PRODUCTO
id_sede	INT	FK → SEDE

cantidad	INT	NOT NULL, DEFAULT 0
fecha_actualizacion	DATETIME	

TRANSFERENCIA_STOCK

Campo	Tipo	Restricciones
id_transferencia	INT	PK, AUTO_INCREMENT
id_producto	INT	FK → PRODUCTO
id_sede_origen	INT	FK → SEDE
id_sede_destino	INT	FK → SEDE
cantidad	INT	NOT NULL, CHECK > 0
fecha	DATETIME	NOT NULL
id_empleado_autoriza	INT	FK → EMPLEADO

TICKET_INCIDENCIA

Campo	Tipo	Restricciones
id_ticket	INT	PK, AUTO_INCREMENT
id_cliente	INT	FK → CLIENTE, NULL si anónimo
id_agente	INT	FK → EMPLEADO
id_pedido	INT	FK → PEDIDO_ONLINE, NULL si consulta general
asunto	VARCHAR(255)	NOT NULL
descripcion	TEXT	
fecha_apertura	DATETIME	NOT NULL
estado	ENUM('abierto','en_gestion','resuelto')	NOT NULL
fecha_cierre	DATETIME	NULL
nota_resolucion	TEXT	NULL

VALORACION

Campo	Tipo	Restricciones
id_valoracion	INT	PK, AUTO_INCREMENT
id_cliente	INT	FK → CLIENTE
id_producto	INT	FK → PRODUCTO
puntuacion	TINYINT	NOT NULL, CHECK 1-5
comentario	TEXT	
fecha	DATETIME	NOT NULL
verificada	TINYINT(1)	NOT NULL, DEFAULT 0
UNIQUE	(id_cliente, id_producto)	Una valoración por cliente/producto

MOVIMIENTO_PUNTOS

Campo	Tipo	Restricciones
id_movimiento	INT	PK, AUTO_INCREMENT
id_cliente	INT	FK → CLIENTE
id_pedido	INT	FK → PEDIDO_ONLINE, NULL si canje manual
puntos	INT	NOT NULL
tipo	ENUM('ganado','canjeado')	NOT NULL

fecha	DATETIME	NOT NULL
descripcion	VARCHAR(255)	

4. Relaciones entre entidades

Relación	Cardinalidad	Descripción
SEDE — EMPLEADO	1:N	Un empleado pertenece a una sede. Una sede tiene varios empleados.
SEDE — STOCK_UBICACION	1:N	Cada sede tiene registros de stock para cada producto.
SEDE — TRANSFERENCIA_STOCK (origen/destino)	1:N	Una sede puede ser origen o destino de múltiples transferencias.
EMPLEADO — PROVEEDOR	1:N	Cada proveedor tiene un representante comercial (empleado) asignado.
EMPLEADO — VENTA_PRESENCIAL	1:N	Un empleado puede realizar múltiples ventas presenciales.
EMPLEADO — TICKET_INCIDENCIA	1:N	Un agente gestiona múltiples tickets de incidencia.
EMPLEADO — TRANSFERENCIA_STOCK	1:N	Un empleado puede autorizar múltiples transferencias.
CATEGORIA — CATEGORIA	1:N (auto-referencia)	Una categoría puede tener subcategorías. La raíz no tiene padre.
CATEGORIA — PRODUCTO	1:N	Un producto pertenece a exactamente una subcategoría.
PRODUCTO — HISTORIAL_PRECIO	1:N	Un producto puede tener múltiples entradas de precio histórico.
PRODUCTO — PROMOCION	1:N	Un producto puede tener múltiples promociones a lo largo del tiempo.
PRODUCTO — STOCK_UBICACION	1:N	Un producto tiene su nivel de stock en cada ubicación.
PRODUCTO — CONDICION_PROVEEDOR	N:M resuelta	Un producto puede ser suministrado por varios proveedores. La tabla CONDICION_PROVEEDOR resuelve esta relación N:M y almacena precio de coste, plazo y fechas.
PROVEEDOR — CONDICION_PROVEEDOR	1:N	Un proveedor puede tener condiciones para múltiples productos.
CLIENTE — DIRECCION	1:N	Un cliente puede tener múltiples direcciones guardadas.
CLIENTE — PEDIDO_ONLINE	1:N	Un cliente puede realizar múltiples pedidos online.
CLIENTE — VENTA_PRESENCIAL	0..1:N	Una venta presencial puede o no estar vinculada a un cliente registrado.
CLIENTE — TICKET_INCIDENCIA	1:N	Un cliente puede abrir múltiples tickets de incidencia.
CLIENTE — VALORACION	N:M resuelta	Un cliente puede valorar múltiples productos. La tabla VALORACION resuelve el N:M con UNIQUE(id_cliente, id_producto).

CLIENTE — MOVIMIENTO_PUNTOS	1:N	Un cliente tiene múltiples movimientos de puntos en su histórico.
PEDIDO_ONLINE — LINEA_PEDIDO	1:N	Un pedido contiene múltiples líneas de pedido.
PEDIDO_ONLINE — ENVIO	1:N	Un pedido puede generar múltiples envíos parciales.
PEDIDO_ONLINE — TICKET_INCIDENCIA	0..1:N	Un ticket puede estar o no vinculado a un pedido.
PEDIDO_ONLINE — MOVIMIENTO_PUNTOS	1:N	Un pedido puede generar movimientos de puntos (ganados y/o canjeados).
LINEA_PEDIDO — PRODUCTO	N:1	Cada línea de pedido referencia un producto.
ENVIO — LINEA_ENVIO	1:N	Un envío contiene las líneas de los productos que incluye.
VENTA_PRESENCIAL — LINEA_VENTA	1:N	Una venta presencial contiene múltiples líneas de venta.
VENTA_PRESENCIAL — DEVOLUCION_PRESENCIAL	1:N	Una venta puede generar una o varias devoluciones.
PRODUCTO — VALORACION	1:N	Un producto puede tener múltiples valoraciones de distintos clientes.

5. Decisiones de diseño justificadas

Tabla CATEGORIA con auto-referencia

Motivo: Lo pide el cliente

El enunciado describe una jerarquía de 3 niveles (Categoría → Subcategoría → Tipo). En lugar de crear tres tablas separadas, se usa una única tabla CATEGORIA con el campo id_categoria_padre que apunta a sí misma. Esto es más flexible y permite cualquier nivel de profundidad sin cambiar el esquema.

LINEA_PEDIDO, LINEA_VENTA, LINEA_ENVIO como tablas independientes

Motivo: Lo propongo yo

Son necesarias para resolver las relaciones N:M entre pedidos/ventas/envíos y productos. Además, almacenan el precio_unitario en el momento de la compra, lo cual es fundamental para la integridad histórica: si el PVP cambia después, las líneas de pedido deben reflejar el precio real que se cobró.

CLIENTE permite valores NULL para anónimos

Motivo: Lo pide el cliente

El enunciado indica que existen clientes que compran en tienda sin registrarse. Se ha optado por incluirlos en la misma tabla CLIENTE con nombre, email y password_hash a NULL y el campo registrado=0. Esto permite la vinculación posterior cuando se registren online.

HISTORIAL_PRECIO separado del PRODUCTO

Motivo: Lo pide el cliente

El PVP actual se mantiene en PRODUCTO para facilitar consultas rápidas, pero cada cambio de precio se registra en HISTORIAL_PRECIO con fecha_inicio y fecha_fin. Así se puede reconstruir el precio que tenía un producto en cualquier momento del pasado.

CONDICION_PROVEEDOR con fechas históricas

Motivo: Lo pide el cliente

El enunciado indica explícitamente que el precio de coste y el plazo de entrega cambian periódicamente y que es importante guardar el histórico. Se añaden fecha_inicio y fecha_fin para poder consultar las condiciones vigentes en cualquier fecha pasada.

MOVIMIENTO_PUNTOS sin campo saldo_actual en CLIENTE

Motivo: Lo propongo yo

El enunciado dice literalmente: 'El saldo actual de puntos de cada cliente tiene que poder calcularse siempre desde el histórico de movimientos'. Esto implica que no se debe almacenar un saldo pre-calculado que pueda desincronizarse. El saldo se obtiene con SUM(puntos) filtrando por tipo en la tabla MOVIMIENTO_PUNTOS.

VALORACION con flag 'verificada'

Motivo: Lo pide el cliente

Marketing quiere filtrar entre valoraciones de compra confirmada y las antiguas sin compra previa. El campo verificada (TINYINT 0/1) resuelve esto sin eliminar el histórico existente.

DEVOLUCION_PRESENCIAL vinculada a VENTA_PRESENCIAL

Motivo: Lo pide el cliente

Las devoluciones en tienda se gestionan con un documento de devolución vinculado al ticket original. Las devoluciones online, en cambio, generan un TICKET_INCIDENCIA, por lo que no se necesita una tabla de devoluciones online separada.

6. Preguntas de reflexión obligatorias

1. El texto menciona que un pedido online puede generar varios envíos. ¿Cómo lo has modelado y por qué?

Se ha modelado mediante una relación 1:N entre PEDIDO_ONLINE y ENVIO. Cada envío tiene su propio número de seguimiento, transportista y fecha estimada de entrega, además de la sede de origen (FK → SEDE).

Para gestionar qué productos van en cada envío y en qué cantidad, se ha creado la tabla

LINEA_ENVIO que vincula cada ENVIO con los PRODUCTO correspondientes e incluye la cantidad.

Este diseño permite que un pedido de, por ejemplo, 3 productos se divida en 2 envíos parciales si el almacén A tiene stock de 2 artículos y el almacén B tiene el tercero. Cada envío refleja exactamente qué contiene y desde dónde sale, sin pérdida de información.

2. ¿Has optado por una sola tabla o dos tablas para ventas presenciales y pedidos online?

Se ha optado por dos tablas separadas: PEDIDO_ONLINE y VENTA_PRESENCIAL.

Ventajas de esta decisión:

- Los procesos son estructuralmente distintos: un pedido online tiene dirección de entrega, envíos y puede generar incidencias por devolución; una venta presencial está vinculada a una sede y un empleado concreto.
- Evita columnas NULL innecesarias: si usáramos una sola tabla, los campos de envío serían NULL para ventas presenciales y la dirección de entrega sería NULL para ventas en tienda.
- Mayor claridad en las consultas y en el mantenimiento del esquema.

Inconvenientes asumidos:

- Para obtener el total de ventas de un producto hay que hacer UNION entre ambas tablas.
- Alguna lógica de negocio (como el historial completo de un cliente) requiere consultar las dos tablas.

Este trade-off es aceptable dado que los procesos son funcionalmente diferentes y el enunciado los describe como tales.

3. ¿Cómo diferencias el histórico de precios del histórico de promociones?

Son dos entidades completamente distintas en el modelo:

HISTORIAL_PRECIO registra los cambios en el PVP base del producto a lo largo del tiempo. Es una decisión interna de precio, no tiene fecha de fin predeterminada y representa el valor real del producto.

PROMOCION registra descuentos porcentuales temporales sobre el PVP. Tiene fecha de inicio y fin definidas, un porcentaje de descuento y puede haber varias simultáneas para el mismo producto (aunque en la práctica se solaparían).

El precio final que paga un cliente se calcula como: $\text{PVP base} \times (1 - \text{descuento_porcentual} / 100)$, consultando si existe una promoción activa en la fecha del pedido. El campo precio_unitario de LINEA_PEDIDO almacena ya el precio final cobrado, preservando la información histórica.

4. ¿Guardarías también un campo saldo_actual en CLIENTE? ¿Por qué sí o por qué no?

No se ha incluido un campo saldo_actual en la tabla CLIENTE, y la decisión está justificada por el propio enunciado: 'El saldo actual de puntos de cada cliente tiene que poder calcularse siempre desde el histórico de movimientos.'

Almacenar un saldo_actual pre-calculado tendría los siguientes riesgos:

- Inconsistencia: si se inserta un movimiento sin actualizar el saldo, el dato estaría desincronizado.
- Redundancia: es información derivable en todo momento con una simple consulta SUM.
- Integridad comprometida: en caso de corrección de un movimiento pasado, habría que recalcular y actualizar el saldo manualmente.

El saldo se obtiene con: `SELECT SUM(CASE WHEN tipo='ganado' THEN puntos ELSE -puntos END) FROM MOVIMIENTO_PUNTOS WHERE id_cliente = ?`

Esta es la aproximación correcta desde el punto de vista de la integridad de datos (event sourcing).

5. Un cliente sin cuenta (compra presencial) puede luego vincularse a una cuenta online. ¿Cómo lo has previsto?

Se ha previsto en el propio diseño de la tabla CLIENTE. Los clientes anónimos (compras presenciales) se almacenan en CLIENTE con el campo registrado=0 y los campos email, password_hash y nombre a NULL. Cada venta presencial puede o no estar vinculada a un id_cliente.

Cuando el cliente se registra online y solicita vincular su historial presencial, el proceso sería:

1. Se crea el nuevo CLIENTE registrado con sus datos completos.
2. Las VENTA_PRESENCIAL que estaban vinculadas al cliente anónimo se actualizan con el nuevo id_cliente.
3. Opcionalmente, el registro anónimo se puede eliminar o marcar como inactivo.

Esta arquitectura permite la vinculación sin pérdida de datos históricos y resuelve el problema mencionado por Laura Pons en el acta de reunión.

6. ¿Hay algún requisito que hayas decidido NO implementar o simplificar?

Se han tomado las siguientes simplificaciones menores:

1. El representante comercial de NexShop asignado a cada proveedor se modela como una FK a EMPLEADO dentro de la tabla PROVEEDOR, sin crear una tabla de asignación histórica.

Justificación: el enunciado no menciona que esta asignación tenga histórico ni que cambie con frecuencia.

2. Las devoluciones online se gestionan como TICKET_INCIDENCIA tal como indica el enunciado, y generarían un envío de recogida (ENVIO con tipo especial). No se ha creado una tabla DEVOLUCION_ONLINE separada porque el flujo descrito pasa íntegramente por atención al cliente.

3. No se ha modelado la entidad 'Departamento' de forma explícita. Los empleados tienen un campo rol y están asignados a una sede, lo cual es suficiente para las consultas requeridas. El enunciado no pide gestionar departamentos como entidad independiente.

Ninguna de estas simplificaciones implica pérdida de funcionalidad para los requisitos explícitos del cliente.